

Seminario de Automatización

Clase 1

Introducción a la automatización. Herramientas y primeros flujos.

Tecnicatura Universitaria en Programación
Gonzalo Bechara Balaldi

Qué se van a llevar

Elegir con criterio entre Zapier, Make, n8n, UiPath o un script propio, y saber por qué

Hacer que sistemas que no se conocen entre sí intercambien información

Sumar un modelo de lenguaje donde corresponde, y reconocer cuándo no

Tendrán una habilidad clave en el ambiente laboral actual

Modelar un proceso como flujo: disparador, datos, nodos, acciones

Integrar APIs sin nodo dedicado, con manejo de errores y aviso de fallas

Poner revisión humana donde la confianza no alcanza

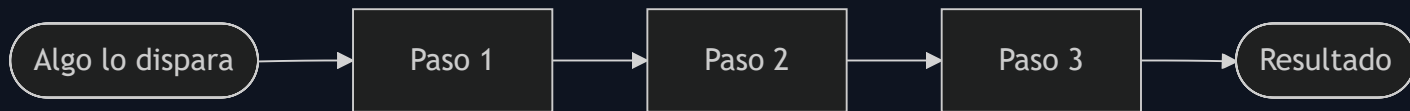
Ejemplos prácticos trasladables a ambientes reales

Qué es automatizar



Qué es automatizar

Pensemos en un proceso



Disparador

Qué hace que el proceso arranque. Una hora, un evento, una persona.

Entrada

Con qué datos empieza y de dónde salen.

Pasos

Qué transformación sufre el dato en cada tramo.

Salida

Qué queda hecho cuando terminó, y cómo se verifica.

Automatizar es escribir este dibujo de manera que se ejecute solo.

Lo que automatizar no es

No es "que la computadora se dé cuenta". Alguien tiene que haber escrito la regla, aunque sea con el mouse.

No es gratis. Tiene costo de construcción, de plataforma y, sobre todo, de mantenimiento.

No es necesariamente inteligencia artificial. La enorme mayoría de lo que vamos a construir es determinista y no tiene un modelo adentro.

No arregla un proceso roto (aunque muchas veces lo evidencia). Un proceso desprolijo automatizado es un proceso desprolijo que ahora falla más rápido.

Tres niveles, según cuánto hay que decidir

- | | | |
|---|---|---------------------------------------|
| 1 | Mecánico. Un proceso que tiene la misma entrada y la misma salida siempre. | copiar una fila a otro sistema |
| 2 | Decisión estructurada. Hay caminos alternativos, pero la regla se puede escribir y la condición se puede verificar. Si pagó, dar acceso. | bifurcar según una regla |
| 3 | Decisión no estructurada. No existe una regla escribible. Requiere criterio, y si la toma un agente de IA, requiere revisión. | clasificar la intención de un mensaje |

El seminario recorre los tres, en ese orden. Hoy estamos en el primero.

¿Cómo se mueve la automatización?

Utiliza la interfaz gráfica

El robot usa la pantalla como la usaría una persona: clics, teclas, lectura de la ventana. Es lo que hace el RPA clásico.

Frágil: cambia un botón de lugar y puede romperse.

Usa la API

El sistema expone un contrato y la automatización lo consume. Es donde vamos a vivir todo el seminario.

Estable: Mantiene la separación de los sistemas, manteniendo alto nivel de optimización.

Altera directamente la DB

Escribir directo en el almacenamiento del otro sistema, saltando su lógica.

Rápido y peligroso: te salteás las validaciones que los desarrolladores pusieron en el back con criterio.

Qué conviene automatizar

Buenos candidatos

- Se repite seguido y siempre igual
- Los pasos se pueden escribir sin ambigüedad
- El error es detectable o reversible
- Las reglas llevan meses sin cambiar
- Hoy lo hace una persona, aburrida

Malos candidatos

- El proceso todavía no está definido
- Cada caso es una excepción distinta
- Corre tres veces por año
- Un error cuesta caro y nadie lo revisa
- El proceso está por cambiar el mes que viene

Antes de automatizar un proceso hay que **entenderlo**, y muy seguido conviene **simplificarlo** primero.

¿Cómo sé si me conviene automatizar? (Con números)

Se gana

tiempo ahorrado por corrida
× cantidad de corridas
× horizonte razonable

+ errores humanos que dejan de pasar

Se paga

tiempo de construirlo
+ costo de la plataforma
+ **mantenimiento**

+ el día que falla y hay que entender por qué

El mantenimiento está resaltado porque es fácil olvidarse de que a una automatización (como todo sistema) hay que mantenerla.

Con qué se automatiza

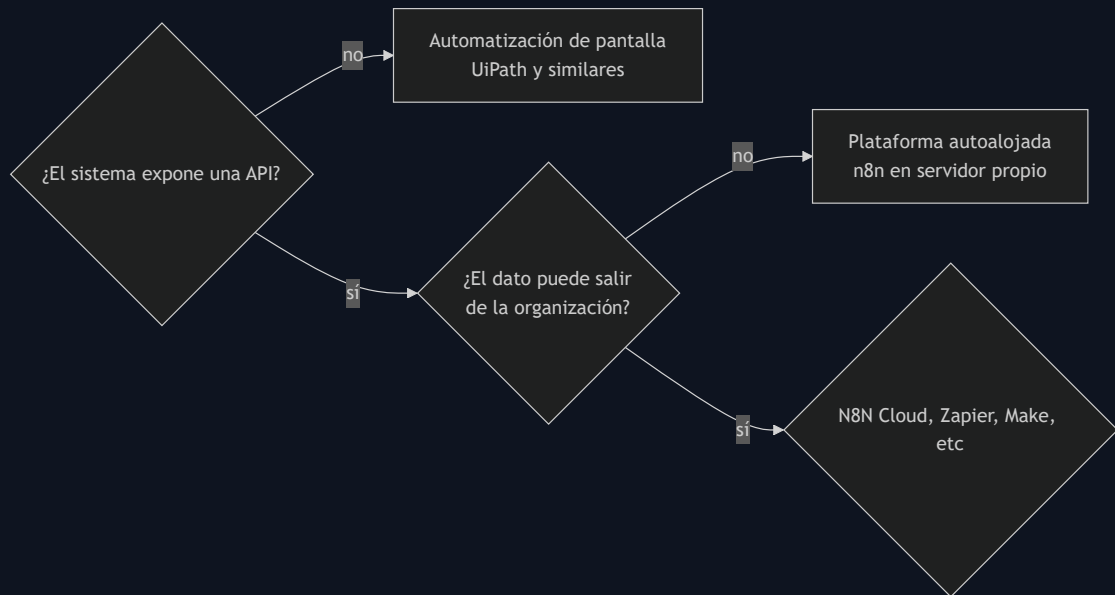
Panorama comparado de herramientas

Cinco enfoques

	Modelo	Dónde corre	Fuerte en	Techo
Zapier	servicio cerrado	nube del proveedor	catálogo enorme de integraciones, cero fricción para empezar	lógica ramificada, costo por tarea ejecutada
Make	servicio cerrado	nube del proveedor	editor visual muy bueno para flujos con ramas	no hay opción de alojarlo uno mismo
n8n	fair-code	nube o servidor propio	control del dato, código embebido, extensible	alguien tiene que operar la instancia
UiPath	RPA	máquina de escritorio	sistemas viejos sin API, automatización de pantalla	frágil ante cambios de interfaz, licenciamiento caro
Script + planificador	código propio	donde quieras	control total, sin límites de la plataforma	todo lo demás lo escribís y lo mantenés vos

Ninguna es mejor que las otras en abstracto. La pregunta correcta es siempre "¿para qué caso?".

Cómo se elige



Un script propio siempre es una respuesta válida. Deja de serlo cuando aparecen la observabilidad, los reintentos y los cinco compañeros que también tienen que poder tocarlo. Conviene este caso para procesos ultra fijos y confiables.

Por qué n8n en este seminario

Se utiliza en el mercado

Es la herramienta más utilizada actualmente para automatizaciones, por equipos técnicos.

Se puede alojar

Que sea open source nos permite que cada uno pueda usar la herramienta sin límites.

La definición es un JSON

Se exporta, se versiona en Git, se revisa en un pull request. Como cualquier código.

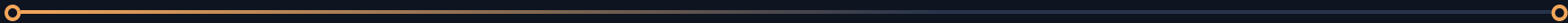
Tiene código adentro

Cuando el nodo no alcanza, se escribe JavaScript. No hay un muro al final del camino.

Los conceptos se transfieren

Disparador, nodo, ítem, credencial y ejecución existen con otro nombre en todas las demás.

$n8n$



Anatomía de un flujo



Workflow

El flujo entero. Un grafo dirigido de nodos conectados.

Nodo

Una operación. Recibe datos, hace algo, entrega datos.

Conexión

Por dónde viaja el dato de un nodo al siguiente.

Los cuatro tipos de nodo

- | | | |
|---|--|---------|
| ▶ | Disparador. Abre el flujo. Manual, programado o por recepción de un pedido externo. Todo flujo tiene exactamente uno. | trigger |
| ▶ | Acción. Habla con un servicio: leer una planilla, mandar un correo, crear un registro. | action |
| ▶ | Lógica. No habla con nadie afuera. Filtra, bifurca, une, arma o renombra campos. | core |
| ▶ | Código. JavaScript propio cuando ningún nodo hace lo que hace falta. | code |

Regla práctica: si un nodo existente resuelve el caso, usarlo. El nodo de código es la salida de emergencia, no el punto de partida.

El dato: items

Entre nodo y nodo no viaja "un dato". Viaja una **lista de items**, y cada ítem tiene su parte estructurada y, si corresponde, su parte binaria.

Un nodo se ejecuta **una vez por cada ítem** que recibe.

De ahí que un solo nodo de correo pueda mandar cincuenta mensajes: recibió cincuenta items.

```
[
  {
    "json": {
      "email": "ana@ejemplo.com",
      "puntaje": 9
    }
  },
  {
    "json": {
      "email": "luis@ejemplo.com",
      "puntaje": 4
    }
  }
]
```

Expresiones

Cualquier campo de configuración puede dejar de ser un valor fijo y pasar a leer datos del flujo.

Ir a leer datos de distintos nodos nos permite que cada repetición de un nodo no sea idéntica a la anterior.

Cuidado con

nombres de campo que cambian entre fuentes, y valores que a veces vienen vacíos.

```
// el item que está procesando este nodo
{{ $json.email }}

// un campo de un nodo anterior, por nombre
{{ $('Traer respuestas').item.json.puntaje }}

// cualquier expresión de JavaScript
{{ $json.nombre.trim().toLowerCase() }}
```

Ejecuciones y credenciales

Ejecución

Una corrida del flujo. Queda registrada con la entrada y la salida de cada nodo.

Manual — la que dispara uno desde el editor, para probar.

De producción — la que dispara el trigger real.

Ese registro es donde vas a vivir cuando algo falle.

Credencial

Las llaves de acceso a los servicios, guardadas aparte del flujo y referenciadas por nombre.

Por eso el JSON exportado se puede subir a un repositorio:
las credenciales no viajan adentro.

Nunca escribir una clave dentro de un nodo. Revisar esto antes de cada entrega.

Caso de uso



El problema

Dos servicios que no se conocen entre sí, y nadie va a escribir un backend para presentarlos.

Servicio A — encuestas

Recibe respuestas de un formulario de satisfacción. Expone las respuestas por API.

Devuelve, entre otros: `email_address`, `first_name`, `last_name`, `score`, `opt_in`.

Servicio B — lista de distribución

Guarda contactos para el envío de novedades. Permite dar de alta por API.

Espera: `email`, `nombre`, `origen`.

No hay integración entre ambos. Hoy alguien exporta una planilla los viernes y carga los contactos a mano.

Alternativas que descartamos

Seguir a mano

Funciona hasta que alguien se toma vacaciones. No escala y se olvida.

Escribir un servicio propio

Semanas de trabajo, un despliegue que mantener y una alerta que atender, para veinte líneas de lógica.

Pedirle la integración al proveedor

No existe y no la van a hacer. Somos un cliente entre miles.

La automatización gana acá por costo de construcción y de cambio, no por elegancia.

Qué hay que resolver

1	Cuándo corre el proceso, y cómo evitar procesar dos veces la misma respuesta	disparador
2	Traer las respuestas nuevas desde el servicio A	lectura
3	Quedarse solo con las que corresponde: Consentimiento dado	filtro
4	Traducir los nombres de los campos, porque cada servicio los llama distinto	mapeo
5	Dar de alta el contacto en el servicio B	escritura

Los pasos 3 y 4 son el objetivo: **homogeneizar** lo que produce un sistema para que lo entienda otro.

Práctica guiada

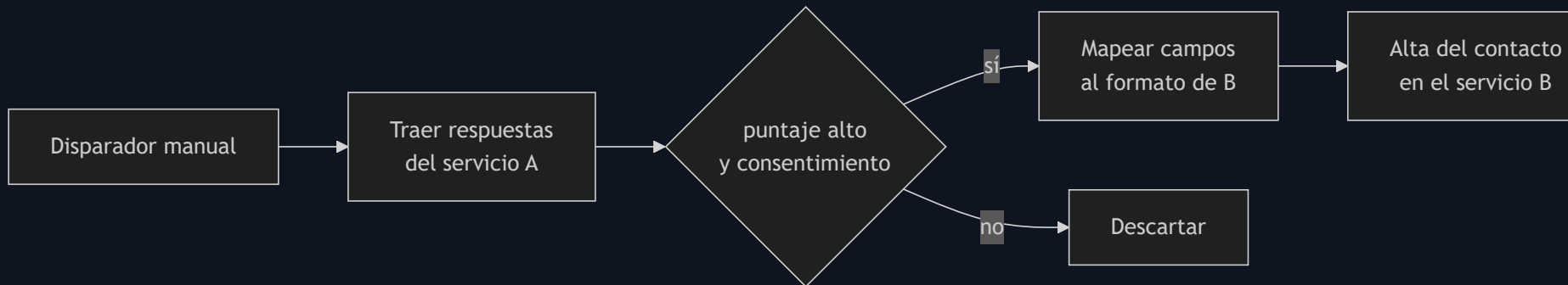


Paso 0 — Entrar a la instancia

- | | | |
|---|--|-------------|
| a | Ingresar a su cuenta con la invitación de su correo | |
| c | Recorrido de la pantalla: lienzo, panel de nodos, panel de ejecuciones, credenciales | orientación |
| b | Crear un flujo vacío, ponerle de nombre su nombre personal y la identificación de la clase | importante |
| d | Crear un flujo vacío y ponerle su nombre y apellido | convención |

La instancia es compartida. Todo lo que hagan queda dentro de su proyecto: no tocar los flujos de otro usuario.

Práctica — Encuesta a lista de distribución



El paso que importa

`email_address` del servicio A tiene que llegar como `email` al servicio B. Ese renombrado es la esencia de la clase.

Cómo saber que salió bien

Consultar el servicio B y encontrar ahí los contactos que corresponden, y solo esos.

Lo que probablemente falle

El campo llega vacío

Casi siempre es un nombre mal escrito en la expresión. La vista previa lo muestra al instante.

El nodo corre una sola vez

Recibió un item con una lista adentro en vez de varios items. Hay que separarlos primero.

El servicio responde 4xx

Leer el cuerpo de la respuesta antes que el código. Suele decir exactamente qué campo falta.

Se dieron de alta contactos de más

La condición del filtro está mirando el campo equivocado, o comparando texto contra número.

Diagnosticar una automatización rota es la mitad del oficio. Lo vamos a practicar todas las clases.

Cierre

Hoy vimos

- Qué es un proceso y cuándo conviene automatizarlo
- Cinco enfoques y con qué criterio se elige
- Flujo, nodo, item, expresión, ejecución y credencial
- Dos servicios ajenos intercambiando información

Qué falló durante la práctica

Repaso en vivo de los errores que aparecieron hoy en el aula.

Para la clase que viene

Exportar la definición del flujo de la práctica en formato JSON y subirla a su propio repositorio. Va a ser el lugar en donde pueden guardar sus proyectos del seminario.

